Day 61 — Syntax, Line by Line

Week 9 · A Many-to-One RNN Classifier on Real Short-Text Sentiment

Watch: Word Embedding and Word2Vec, Clearly Explained!!! (StatQuest with Josh Starmer) | L15.7 An RNN Sentiment Classifier in PyTorch (Sebastian Raschka)

This guide walks through `day61_many_to_one_sentiment.py` in the order the code appears. The code boxes are cut from the real script by program (inline comments removed so lines fit); the notes under each box explain the Python and NumPy used. The companion *complete guide* covers the experiments and results.

1. Setup

Imports and constants

CODE

```
import copy
import os
import re
from collections import Counter
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
np.random.seed(42)
DATA_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)),
    "sentiment_labelled_sentences")
PAD, UNK = 0, 1
MAX_LEN = 40
```

- `import copy` — gives **copy.deepcopy**, used to keep a full, independent snapshot of the best network.
- `import os` — path tools, so the dataset folder is found next to the script wherever it is run from.
- `import re` — regular expressions: pattern matching on text, used by the tokenizer.
- `from collections import Counter` — a dictionary that counts things; here, how often each word appears.
- `matplotlib.use("Agg")` — draw plots straight to image files, with no window. Must come before importing pyplot.
- `os.path.abspath(__file__)` — the full path of this script. **dirname** strips the file name, and **os.path.join** glues on the folder name with the right slash for the operating system.
- `PAD, UNK = 0, 1` — two names assigned at once (tuple unpacking). Id 0 means padding, id 1 means unknown word.
- `MAX_LEN = 40` — a constant in capitals by convention: sentences are cut and padded to 40 tokens.

2. The text pipeline

load_sentences — reading the three files

CODE

```
def load_sentences():
    texts, labels, sources = [], [], []
    for name in ["amazon_cells", "imdb", "yelp"]:
        with open(os.path.join(DATA_DIR, name + "_labelled.txt"), encoding="utf-8") as f:
            for line in f:
                text, label = line.rstrip("\n").rsplit("\t", 1)
                texts.append(text); labels.append(int(label)); sources.append(name.split("_")[0])
    return texts, np.array(labels), np.array(sources)
```

- `texts, labels, sources = [], [], []` — three empty lists created in one line.
- `with open(..., encoding="utf-8") as f:` — open the file and close it automatically when the block ends. utf-8 is stated so accented characters read the same on every computer.
- `for line in f:` — a file object can be looped over directly; each turn gives one line of text.
- `line.rstrip("\n")` — remove the newline character from the right end of the line.
- `.rsplit("\t", 1)` — split on a tab, starting from the **right**, at most **once**. A sentence that itself contains a tab is still split correctly into (sentence, label).
- `text, label = ...` — unpack the two pieces into two names.
- `int(label)` — turn the text "0" or "1" into a number.
- `name.split("_")[0]` — cut "amazon_cells" at underscores and keep the first piece, "amazon".
- `np.array(labels)` — list to NumPy array, so it can be indexed with a whole array of positions later.

tokenize — one regular expression

CODE

```
def tokenize(sentence):
    return re.findall(r"[a-z]+(?:'[a-z]+)?|[!?]", sentence.lower())
```

- `sentence.lower()` — make everything lowercase, so *Great* and *great* are the same word.
- `re.findall(pattern, text)` — return a list of every piece of text that matches the pattern, left to right.
- `r"..."` — a raw string: backslashes are passed to the regex engine unchanged.
- `[a-z]+` — one or more letters a to z: a word.
- `(?:'[a-z]+)?` — optionally followed by an apostrophe and more letters, so *don't* stays one token. **(?:)** groups without capturing; the trailing **?** means “zero or one time”.
- `|[!?]` — **|** means “or”: a single ! or ? is also a token. Everything else (digits, commas, full stops) matches nothing and is dropped.

build_vocab — choosing which words get an id

CODE

```
def build_vocab(token_lists, min_count=2):
    counts = Counter(w for toks in token_lists for w in toks)
    itos = ["<pad>", "<unk>"] + sorted(w for w, c in counts.items() if c >= min_count)
    return itos, {w: i for i, w in enumerate(itos)}
```

- `min_count=2` — a default argument: used when the caller does not pass one.
- `Counter(w for toks in token_lists for w in toks)` — a generator with two loops, read left to right: for each sentence, for each word in it. Counter tallies every word.
- `counts.items()` — pairs of (word, count).
- `sorted(w for w, c in ... if c >= min_count)` — keep words seen at least twice, in alphabetical order, so ids are the same on every run.
- `["<pad>", "<unk>"] + sorted(...)` — list concatenation: the two special tokens take ids 0 and 1.
- `itos` — “index to string”: a list, so **itos[5]** is the word with id 5.
- `{w: i for i, w in enumerate(itos)}` — a dictionary comprehension. **enumerate** yields (position, item); the result maps word to id (“stoi”, string to index).

encode — token lists to one padded array

CODE

```
def encode(token_lists, stoi, max_len=MAX_LEN):
    ids = np.full((len(token_lists), max_len), PAD, dtype=int)
    lens = np.zeros(len(token_lists), dtype=int)
    for row, toks in enumerate(token_lists):
        seq = [stoi.get(w, UNK) for w in toks][:max_len] or [UNK]
        ids[row, :len(seq)] = seq
        lens[row] = len(seq)
    return ids, lens
```

- `np.full((n, max_len), PAD, dtype=int)` — an n-by-40 array filled with 0 (padding), holding integers.
- `np.zeros(n, dtype=int)` — one length per sentence, starting at 0.
- `for row, toks in enumerate(token_lists):` — row number and token list together.
- `stoi.get(w, UNK)` — dictionary lookup with a fallback: the word's id, or 1 if the word is not in the vocabulary.
- `[...][:max_len]` — slice: keep at most the first 40 ids.
- `... or [UNK]` — an empty list counts as False in Python, so **or** replaces it with [1]. No sentence ends up empty.
- `ids[row, :len(seq)] = seq` — write the ids into the start of the row; the rest of the row stays 0.
- `return ids, lens` — two values returned as a tuple.

shuffle_within — scrambling words but not padding

CODE

```
def shuffle_within(ids, lens, seed):
    rng = np.random.RandomState(seed)
    out = ids.copy()
    for row in range(len(lens)):
        out[row, :lens[row]] = rng.permutation(ids[row, :lens[row]])
    return out
```

- `np.random.RandomState(seed)` — a private random generator: the same seed always gives the same shuffle.
- `ids.copy()` — work on a copy so the caller's array is left untouched.
- `ids[row, :lens[row]]` — only the real words of this row.
- `rng.permutation(...)` — the same values in random order, as a new array.
- `out[row, :lens[row]] = ...` — write the shuffled words back into the same positions; padding is never moved.

3. Embedding and the masked RNN

embedding_forward and embedding_backward

CODE

```
def embedding_forward(E, ids):
    return E[ids]
def embedding_backward(dX, ids, vocab_size):
    dE = np.zeros((vocab_size, dX.shape[-1]))
    np.add.at(dE, ids, dX)
    return dE
```

- `E[ids]` — fancy indexing: **ids** is an (m, T) array of integers, so the result has shape (m, T, D) — row ids[i, t] of E at every position.

- `dX.shape[-1]` — the size of the last axis: the embedding dimension D.
- `np.zeros((vocab_size, D))` — an empty gradient table, same shape as E.
- `np.add.at(dE, ids, dX)` — for every position, add dX[i, t] into row ids[i, t] of dE. Unlike **dE[ids] += dX**, it adds **every** time an index repeats. That is what makes a word used twice receive both gradients.

rnn_forward_masked

CODE

```
def rnn_forward_masked(X, mask, h0, Wxh, Whh, bh):
    m, T, _ = X.shape
    H = np.zeros((m, T, Whh.shape[0]))
    h = h0
    for t in range(T):
        h_new = np.tanh(X[:, t, :] @ Wxh + h @ Whh + bh)
        m_t = mask[:, t:t + 1]
        h = m_t * h_new + (1.0 - m_t) * h
        H[:, t, :] = h
    return H
```

- `m, T, _ = X.shape` — unpack three sizes; `_` is a name for a value that will not be used.
- `Whh.shape[0]` — the hidden size, read from the recurrent weight matrix.
- `X[:, t, :]` — the word vectors at position t for the whole batch, shape (m, D).
- `X[:, t, :] @ Wxh + h @ Whh + bh` — `@` is matrix multiplication. The bias (shape H) is broadcast across the batch.
- `mask[:, t:t + 1]` — slicing with **t:t+1** keeps the axis, giving shape (m, 1) instead of (m,), so it broadcasts across the hidden units.
- `m_t * h_new + (1.0 - m_t) * h` — a per-row switch: where m_t is 1 take the new state, where it is 0 keep the old one.
- `H[:, t, :] = h` — store this step's state for the backward pass.

rnn_backward_masked

CODE

```
def rnn_backward_masked(dH_direct, dh_last, X, mask, H, h0, Wxh, Whh):
    m, T, _ = X.shape
    dWxh = np.zeros_like(Wxh); dWhh = np.zeros_like(Whh); dbh = np.zeros(Whh.shape[0])
    dX = np.zeros_like(X)
    dh = dh_last
    for t in reversed(range(T)):
        dh = dh + dH_direct[:, t, :]
        m_t = mask[:, t:t + 1]
        h_prev = H[:, t - 1, :] if t > 0 else h0
        da = dh * m_t * (1.0 - H[:, t, :] ** 2)
        dWxh += X[:, t, :].T @ da / m
        dWhh += h_prev.T @ da / m
        dbh += da.sum(axis=0) / m
        dX[:, t, :] = da @ Wxh.T / m
        dh = da @ Whh.T + dh * (1.0 - m_t)
    return dWxh, dWhh, dbh, dX
```

- `np.zeros_like(Wxh)` — zeros with the same shape as Wxh.
- `a = ...; b = ...; c = ...` — semicolons put several short statements on one line.
- `reversed(range(T))` — T-1, T-2, ..., 0: the timesteps in backward order.
- `dh = dh + dH_direct[:, t, :]` — add any gradient sent straight to this step's state (non-zero only for the mean readout). Written as **dh = dh + ...**, not **+=**, so the caller's array is never modified.
- `H[:, t - 1, :] if t > 0 else h0` — a one-line if/else: the previous state, or the initial state at step 0.

- `dh * m_t * (1.0 - H[:, t, :] ** 2)` — the tanh derivative is 1 - tanh squared; multiplying by m_t zeroes it on padding.
- `X[:, t, :].T @ da / m` — **.T** transposes. (D, m) @ (m, H) gives the (D, H) gradient for Wxh, averaged over the batch.
- `dWxh += ...` — accumulate: the same weights are used at every step, so their gradients add up (Day 58).
- `da.sum(axis=0)` — sum over the batch axis, leaving one value per hidden unit.
- `dX[:, t, :] = da @ Wxh.T / m` — the gradient for this position's word vector, later routed into the embedding table.
- `da @ Whh.T + dh * (1.0 - m_t)` — real word: gradient goes back through Whh. Padding: da is zero, so the gradient is passed through unchanged.

4. The classifier

SentimentRNN.__init__

CODE

```
class SentimentRNN:
    def __init__(self, vocab_size, embed_dim=16, hidden_dim=16, lr=0.1, clip=1.0, readout="last",
        seed=0):
        rng = np.random.RandomState(seed)
        self.vocab_size, self.hidden_dim = vocab_size, hidden_dim
        self.lr, self.clip, self.readout = lr, clip, readout
        self.E = rng.randn(vocab_size, embed_dim) * 0.1
        self.Wxh = rng.randn(embed_dim, hidden_dim) * np.sqrt(1.0 / embed_dim)
        self.Whh = rng.randn(hidden_dim, hidden_dim) * np.sqrt(1.0 / hidden_dim)
        self.bh = np.zeros(hidden_dim)
        self.Why = rng.randn(hidden_dim, 2) * np.sqrt(1.0 / hidden_dim)
        self.by = np.zeros(2)
```

- `class SentimentRNN:` — defines a new type that bundles the weights with the functions that use them.
- `def __init__(self, ...)` — runs when an object is created. **self** is the object being built.
- `readout="last"` — a string option with a default value.
- `self.a, self.b = x, y` — store several settings on the object in one line.
- `rng.randn(vocab_size, embed_dim) * 0.1` — normally distributed random numbers, scaled small: the starting embedding table.
- `np.sqrt(1.0 / embed_dim)` — a starting scale that keeps the size of signals roughly steady through a tanh layer.

SentimentRNN.forward

CODE

```
def forward(self, ids, lens, use_mask=True):
    m, T = ids.shape
    self.ids, self.lens = ids, lens
    self.mask = (np.arange(T)[None, :] < lens[:, None]).astype(float) if use_mask else np.ones((m,
        T))
    self.h0 = np.zeros((m, self.hidden_dim))
    self.X = embedding_forward(self.E, ids)
    self.H = rnn_forward_masked(self.X, self.mask, self.h0, self.Wxh, self.Whh, self.bh)
    if self.readout == "last":
        self.feat = self.H[:, -1, :]
    else:
        self.feat = (self.H * self.mask[:, :, None]).sum(axis=1) / lens[:, None]
    self.probs = softmax(self.feat @ self.Why + self.by)
    return self.probs
```

- `np.arange(T)[None, :]` — 0..T-1 as a row. **[None, :]** adds an axis: shape (1, T).
- `lens[:, None]` — the lengths as a column, shape (m, 1).
- `np.arange(T)[None, :] < lens[:, None]` — broadcasting compares every position with every length, giving an (m, T) table of True/False: True where the position is before the sentence's end.
- `.astype(float)` — True/False to 1.0/0.0.
- `A if use_mask else B` — choose the real mask, or all ones when masking is switched off.
- `self.H[:, -1, :]` — index -1 is the last position: the final hidden state of every sentence.
- `self.mask[:, :, None]` — add a trailing axis so the (m, T) mask multiplies the (m, T, H) states.
- `.sum(axis=1) / lens[:, None]` — sum over time, divide by the real length: the average over real words only.
- `softmax(self.feat @ self.Why + self.by)` — one row of two probabilities per sentence.

SentimentRNN.gradients and train_step

CODE

```
def gradients(self, y):
    m, T = self.ids.shape
    dlogits = self.probs - one_hot(y, 2)
    grads = dict(Why=self.feat.T @ dlogits / m, by=dlogits.sum(axis=0) / m)
    dfeat = dlogits @ self.Why.T
    if self.readout == "last":
        dH_direct, dh_last = np.zeros_like(self.H), dfeat
    else:
        dH_direct = dfeat[:, None, :] * self.mask[:, :, None] / self.lens[:, None, None]
        dh_last = np.zeros_like(dfeat)
    grads["Wxh"], grads["Whh"], grads["bh"], dX = rnn_backward_masked(
        dH_direct, dh_last, self.X, self.mask, self.H, self.h0, self.Wxh, self.Whh)
    grads["E"] = embedding_backward(dX, self.ids, self.vocab_size)
    return grads
def train_step(self, ids, lens, y):
    loss = cross_entropy_loss(self.forward(ids, lens), one_hot(y, 2))
    grads, _, scale = clip_by_global_norm(self.gradients(y), self.clip)
    for name, g in grads.items():
        getattr(self, name)[...] -= self.lr * g
    return loss, scale < 1.0
```

- `dict(Why=..., by=...)` — build a dictionary using keyword arguments as the keys.
- `self.feat.T @ dlogits / m` — (H, m) @ (m, 2): the gradient for the output weights.
- `dfeat[:, None, :]` — insert a time axis, (m, 1, H), so the same gradient is shared out to every timestep.
- `self.lens[:, None, None]` — lengths shaped (m, 1, 1) to divide an (m, T, H) array row by row.
- `a, b, c, d = rnn_backward_masked(...)` — a function call split over two lines inside its brackets; four results unpacked.
- `grads["E"] = ...` — add one more entry to the dictionary.
- `grads, _, scale = clip_by_global_norm(...)` — three values come back; the middle one (the norm) is not needed here.
- `for name, g in grads.items():` — loop over (parameter name, gradient) pairs.
- `getattr(self, name)` — fetch an attribute by its name as text: getattr(self, "Whh") is self.Whh.
- `[...] -= self.lr * g` — **[...]** means “all elements”: update the existing array in place.
- `return loss, scale < 1.0` — the comparison gives True when clipping happened.

5. Training

fit — training with a record of every epoch

CODE

```
def fit(data, vocab_size, seed, order="original", epochs=40, batch_size=32, **net_kwargs):
    (ids_tr, len_tr, y_tr), (ids_va, len_va, y_va), (ids_te, len_te, y_te) = data
    net = SentimentRNN(vocab_size, seed=seed, **net_kwargs)
    rng = np.random.RandomState(seed)
    hist = dict(loss=[], train=[], val=[], test=[], clipped=[])
    best_val, best_net, best_epoch = -1.0, None, -1
    for epoch in range(epochs):
        if order == "original":
            src = ids_tr
        elif order == "shuffled once":
            src = shuffle_within(ids_tr, len_tr, seed=1000)
        else:
            src = shuffle_within(ids_tr, len_tr, seed=1000 + epoch)
        perm = rng.permutation(len(y_tr))
        losses, clips = [], []
        for start in range(0, len(perm), batch_size):
            idx = perm[start:start + batch_size]
            T_batch = len_tr[idx].max()
            loss, was_clipped = net.train_step(src[idx][:, :T_batch], len_tr[idx], y_tr[idx])
            losses.append(loss); clips.append(was_clipped)
        hist["loss"].append(np.mean(losses)); hist["clipped"].append(np.mean(clips))
        hist["train"].append(net.accuracy(ids_tr, len_tr, y_tr))
        hist["val"].append(net.accuracy(ids_va, len_va, y_va))
        hist["test"].append(net.accuracy(ids_te, len_te, y_te))
        if hist["val"][-1] > best_val:
            best_val, best_net, best_epoch = hist["val"][-1], copy.deepcopy(net), epoch
    return best_net, best_epoch, {k: np.array(v) for k, v in hist.items()}
```

- `**net_kwargs` — collect any extra keyword arguments into a dictionary, then pass them on with **SentimentRNN(..., **net_kwargs)**.
- `(a, b, c), (d, e, f), (g, h, i) = data` — nested unpacking of three (ids, lens, y) tuples.
- `dict(loss=[], train=[], ...)` — a dictionary of empty lists, one list per thing recorded.
- `if / elif / else on order` — pick this epoch's training sentences: as written, one fixed scramble, or a new scramble.
- `seed=1000 + epoch` — a different seed each epoch gives a different scramble each epoch.
- `rng.permutation(len(y_tr))` — the numbers 0..n-1 in random order: this epoch's sentence order.
- `range(0, len(perm), batch_size)` — 0, 32, 64, ...: the start of each mini-batch.
- `perm[start:start + batch_size]` — up to 32 sentence positions; a slice past the end is simply shorter.
- `len_tr[idx].max()` — the longest sentence in this batch.
- `src[idx][:, :T_batch]` — pick the batch rows, then keep only the first T_batch columns: less padding, less work.
- `np.mean(clips)` — the mean of True/False values is the fraction that are True.
- `hist["val"][-1]` — the newest validation accuracy.
- `copy.deepcopy(net)` — a complete independent copy, so later training does not change the snapshot.
- `{k: np.array(v) for k, v in hist.items()}` — turn every list in the record into an array.

fit_cached — do not train the same run twice

CODE

```
RUNS = {}
def fit_cached(data, vocab_size, seed, order="original", **net_kwargs):
    key = (seed, order, tuple(sorted(net_kwargs.items())))
    if key not in RUNS:
        RUNS[key] = fit(data, vocab_size, seed, order, **net_kwargs)
    return RUNS[key]
```

- `RUNS = {}` — a module-level dictionary that lives for the whole run of the script.
- `tuple(sorted(net_kwargs.items()))` — dictionaries cannot be dictionary keys, so the settings are turned into a sorted tuple of (name, value) pairs.
- `if key not in RUNS:` — train only the first time these exact settings are requested.
- `return RUNS[key]` — every later request gets the stored result. Sections 4, 5 and 6 share runs this way.

bag_of_words and train_bow — the baseline

CODE

```
def bag_of_words(ids, vocab_size):
    X = np.zeros((ids.shape[0], vocab_size))
    rows = np.repeat(np.arange(ids.shape[0]), ids.shape[1])
    X[rows, ids.ravel()] = 1.0
    X[:, PAD] = 0.0
    return X
def train_bow(X_tr, y_tr, X_va, y_va, lr=2.0, epochs=600, l2=1e-4):
    W = np.zeros((X_tr.shape[1], 2)); b = np.zeros(2)
    best = (-1.0, None, None, -1)
    for epoch in range(epochs):
        dlogits = (softmax(X_tr @ W + b) - one_hot(y_tr, 2)) / len(y_tr)
        W -= lr * (X_tr.T @ dlogits + l2 * W)
        b -= lr * dlogits.sum(axis=0)
        val = ((X_va @ W + b).argmax(axis=1) == y_va).mean()
        if val > best[0]:
            best = (val, W.copy(), b.copy(), epoch)
    return best
```

- `np.repeat(np.arange(n), T)` — 0,0,...,0,1,1,...: the row number of every (sentence, position) pair.
- `ids.ravel()` — the (n, T) ids flattened into one long row, lined up with **rows**.
- `X[rows, ids.ravel()] = 1.0` — set one cell per pair. A word appearing twice just sets the same cell twice, so the feature is presence, not count.
- `X[:, PAD] = 0.0` — clear the whole padding column.
- `best = (-1.0, None, None, -1)` — a tuple holding the best result so far.
- `X_tr.T @ dlogits + l2 * W` — the gradient of the loss plus the gradient of the L2 penalty.
- `.argmax(axis=1)` — the position of the largest value in each row: the predicted class.
- `(pred == y_va).mean()` — fraction of matches: accuracy.
- `W.copy()` — a copy is needed because W keeps changing in later epochs.

6. Checks and experiments

The gradient check loop (inside demo_checks)

CODE

```
for name in ["E", "Wxh", "Whh", "bh", "Why", "by"]:
    P = getattr(net, name)
    numeric = np.zeros_like(P)
    for pos in np.ndindex(P.shape):
        old = P[pos]
        P[pos] = old + 1e-5; lp = cross_entropy_loss(net.forward(ids, lens), one_hot(y, 2))
        P[pos] = old - 1e-5; lm = cross_entropy_loss(net.forward(ids, lens), one_hot(y, 2))
        P[pos] = old
        numeric[pos] = (lp - lm) / 2e-5
    errs.append(np.max(np.abs(numeric - analytic[name]) / (np.abs(numeric) +
        np.abs(analytic[name]) + 1e-12)))
```

- `P = getattr(net, name)` — P is the parameter array itself, not a copy: changing P changes the network.
- `np.ndindex(P.shape)` — yields every index of the array as a tuple, whatever its number of dimensions.
- `P[pos] = old + 1e-5` — nudge one single weight up by a tiny amount.
- `(lp - lm) / 2e-5` — the central difference: the measured slope of the loss with respect to that weight.
- `P[pos] = old` — put the weight back before moving to the next one.
- `np.abs(a - b) / (np.abs(a) + np.abs(b) + 1e-12)` — relative error; the tiny constant avoids dividing zero by zero.
- `np.max(...)` — the worst entry of this parameter.

Padding a test batch by hand (inside demo_checks)

CODE

```
ids = rng.randint(2, 12, size=(4, 5))
ids[0, 3] = ids[0, 1]
ids[np.arange(5)[None, :] >= lens[:, None]] = PAD
```

- `rng.randint(2, 12, size=(4, 5))` — random integers from 2 to 11: real word ids, never 0 or 1.
- `ids[0, 3] = ids[0, 1]` — force one word to appear twice in the first sentence.
- `ids[boolean_array] = PAD` — boolean indexing: set every cell where the condition is True. The condition is “position is at or past this sentence's length”.

The shuffle test (inside demo_word_order)

CODE

```
p0 = net.predict(ids_te, len_te)
ps = [net.predict(shuffle_within(ids_te, len_te, s), len_te) for s in range(5)]
val.append(h["val"][be]); test.append((p0 == y_te).mean())
shuf.append(np.mean([(p == y_te).mean() for p in ps]))
flip.append(np.mean([(p != p0).mean() for p in ps]))
```

- `[f(s) for s in range(5)]` — a list comprehension: five sets of predictions, one per shuffle seed.
- `(p == y_te).mean()` — accuracy of one set of predictions.
- `(p != p0).mean()` — the fraction of sentences whose prediction changed after shuffling.
- `np.mean([... for p in ps])` — average over the five shuffles.

Choosing the learning rate on validation (inside demo_readout)

CODE

```
best = {r: max([k for k in rows if k[0] == r and k[2] == 1.0], key=lambda k:
    rows[k]["val"].mean()) for r in ["last", "mean"]}
```

- `{r: ... for r in ["last", "mean"]}` — a dictionary with one entry per readout.
- `[k for k in rows if k[0] == r and k[2] == 1.0]` — the settings for this readout that used clipping.
- `max(items, key=lambda k: ...)` — the item with the largest value of the key function. **lambda** is a small unnamed function; here it returns the mean validation accuracy.
- `rows[k]["val"].mean()` — validation, not test: the choice never looks at test accuracy.

7. Patterns worth remembering

- `E[ids]` looks rows up; `np.add.at(dE, ids, dX)` is its correct backward.
- `np.arange(T)[None, :] < lens[:, None]` builds a padding mask with no loop.
- `m_t * new + (1 - m_t) * old` is a per-row switch that works on whole batches.
- `x[:, t:t + 1]` keeps an axis that `x[:, t]` would drop.
- `getattr(self, name)[...] -= ...` updates a weight chosen by name, in place.
- `copy.deepcopy(net)` snapshots the best model; pick it on validation, report it on test.
- `stoi.get(word, UNK)` handles words the model has never seen.